

Machine Learning (HWS25)

Assignment 4: Latent Variable Models

The archive provided to you contains this assignment description, a dataset in binary format, as well as Python code fragments for you to complete. Comments and documentation in the code provide further information.

Please **adhere to the following guidelines** in all the assignments. If you do not follow those guidelines, we may grade your solution as a FAIL. Provide a single ZIP archive with name `ml25-a0<assignment number>-<your ILIAS login>.zip`. The archive needs to contain:

- A **single PDF report** `a0<assignment number>-solution.pdf` that contains answers to the tasks specified in the assignment, including helpful figures and a high-level description of your approach. **Do not simply convert your Jupyter notebook to a PDF!** Write a separate document, stay focused and brief. Your report **must not exceed 8 pages, excluding references and appendix**. You must use the template `a0<assignment number>-solution.tex` provided to you and you must **fill out the declaration of AI usage** at its end.
- All **the code that you created** and used in its original format and using its original filenames.
- A **PDF document that renders your Jupyter notebook with all figures**.

Further guidelines are:

- A **high quality report is required to achieve an EXCELLENT grade**. Such a report is self-explanatory (i.e. do not refer to your code except for implementation-only tasks), follows good scientific practice (e.g. when using images, tables or citations), does not include hand-written notes, and does not exceed 8 pages. In addition, label all figures (and refer to figure labels in your write-up), include references if you used additional sources or material, and use the tasks numbers of the assignments as your section and subsection numbers.
- You **must work on this assignment in pairs**—i.e, with one (and only one) additional student—and then hand in a pair submission. To do so:
 - The PDF report and notebook must clearly report then **name and ILIAS login of both students** right at the beginning.
 - **Both students must submit** the same assignment on ILIAS separately.
 - You may change with whom your form a pair from assignment to assignment.

To be clear, if only one student of the pair submits the assignment, then only this student will receive the grade.

- Hand-in your solution via ILIAS until the date specified there. This is a hard deadline.

Report. The following list of tasks are pure implementation tasks:

Task 1c, Task 2c

You do *not* need to discuss these tasks in your report.

1 Probabilistic PCA

- a) Study the PPCA generator provided to you (`ppca_gen`). Generate and plot the `toy_ppca` dataset. What is shown in the plot? Vary the amount of noise (σ^2), replot, and describe how the data distribution changes (visually) depending on σ^2 .
- b) Implement MLE for PPCA by completing `ppca_mle`. Only use standard matrix/vector operations and the `svd` function. If we fit the PPCA model with $Z = 2$ on `toy_ppca`, we obtain $\hat{\sigma}_{\text{MLE}}^2 = 0$. Why is this?
- c) Implement the computation of the negative log-likelihood of a given dataset for a given PPCA model. To do so, complete `ppca_nll`.
- d) Load the `secret_ppca` dataset, which was produced by sampling a PPCA model. Discover its secret: How many latent variables have been used (Z)? Do this by
 - (i) Studying the scree plot.
 - (ii) Using validation data.

Do the results agree?

2 Gaussian Mixture Models

- a) Study the GMM generator provided to you (`gmm_gen`). Generate and plot the `toy_ppca` dataset. Describe what is shown in the plot.
- b) Compute a K -Means clustering of `toy_ppca` using $K = 5$ (code provided). Plot the resulting clustering and discuss. Did you expect this result?
- c) Implement the E step and the M step of fitting a GMM with MLE by completing `gmm_e` and `gmm_m`. Only use standard matrix/vector operations and (optionally) the functions mentioned in the hints below.

Hint (E step). Start by computing and verifying $\mathbf{P}$ ($[\mathbf{P}]_{ik} = p_k(\mathbf{x}_i|\boldsymbol{\theta})$ in lecture notation). You can compute the density of each data point (row) in a dataset ($\mathbf{X}$) under multivariate normal distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ using:

```
dist = scipy.stats.multivariate_normal(mean=mu, cov=Sigma)
densities = dist.pdf(X)
```

Hint (M step). Start by computing and verifying $\boldsymbol{\pi}$, then the $\boldsymbol{\mu}_k$'s, then the $\boldsymbol{\Sigma}_k$'s. Perhaps helpful: NumPy's covariance function `np.cov` supports weighted data points (set `aweights` argument accordingly; additionally set `dof=0`).

- d) Fit the GMM model with $K = 5$ using `gmm_fit` (provided). Assign each data point to its most likely component to obtain a clustering and plot the dataset as in subtask b). Compare the resulting clustering with the result of K -Means clustering of subtask b).
- e) Repeat subtask d) with $K = 4$ and with $K = 6$. In both cases, repeat multiple times. Discuss your findings.
- f) Optional: Load the `secret_gmm` dataset, which was produced by sampling a GMM model. Discover it's secret: How many components (K) have been used? Briefly justify your answer.